

Intractable Problems

P: Problems solvable in poly time (sorting, the shortest path)

NP: A solution can be verified in poly time but not the algorithm (ALL P ARE NP, but NOT ALL NP ARE P)

NP-HARD: Problem at least as difficult as hardest problem in NP

NP-COMPLETE: Problem both in NP and NP-hard

All NP-COMPLETE are NP-hard, NP-COMPLETE is a subset of NP-hard problems with added constraint that they must be in NP (sols can be verified in polynomial time)

Polynomial Reductions:

Polynomial reductions: Transformation from problem U to V that:

1. MAPS YES instances to YES instances and NO instances to NO instances
2. Can be computed in poly time

Showing a problem is NP-Complete: Prove it's in NP and reduce a known NP-complete to it.

E.g.

- 3SAT to Vertex Cover - creating a graph from a boolean formula
- Vertex Cover to Clique: Using the complement of a graph

Strategies:

1. Superpolynomial Exact algorithm
 - Meet in the middle (e.g. Subset Sum): Reduce $O(2^n)$ to $O(2^{n/2})$ time
 - Dynamic programming (e.g. TSP): Reduce $O(n!)$ to $O(2^{nn^2})$ time
2. Approximation Algorithms
 - Vertex Cover: 2-approximation by greedily selecting edge endpoints
 - Metric TSP: 2-approximation using minimum spanning tree and shortcuts
3. Other approaches
 - Fixed-param algorithms
 - Heuristics
 - Solving on restricted inputs

1.NP-Completeness Proofs

Question type: Prove that a problem X is NP-complete. How to solve:

- First show X is in NP by describing a certificate and verification algorithm
- Then reduce a known NP-complete problem (like 3SAT) to X using a polynomial-time transformation
- Prove correctness by showing YES instances map to YES instances and vice versa
- Analyze time complexity to confirm polynomial runtime of the reduction

2. Polynomial Reductions

Question type: Give a polynomial reduction from problem A to problem B. How to solve:

- Design a transformation f that maps instances of A to instances of B
- Prove f preserves answers: x is a YES instance of A if and only if $f(x)$ is a YES instance of B
- Analyze the runtime to ensure f is computable in polynomial time
- Use clear diagrams to illustrate the transformation when appropriate

3. Approximation Algorithms

Question type: Design a c-approximation algorithm for problem X. How to solve:

- Develop an algorithm that runs in polynomial time
- Prove it always produces a valid solution
- Establish that the solution is at most c times worse than optimal (for minimization problems)
- Show tightness by finding an instance where the approximation ratio is exactly c

4. Exponential-Time Algorithms

Question type: Design an algorithm better than brute force for problem X. How to solve:

- Apply techniques like meet-in-the-middle or dynamic programming
- Prove correctness of your algorithm
- Analyze time complexity to show improvement over naive approach
- Discuss space-time tradeoffs if relevant

5. Analysis of Coping Strategies

Question type: Which coping strategy would be most appropriate for problem X? How to solve:

- Analyze the problem structure and constraints
- Consider approximation ratio needs (is a 2-approximation good enough?)
- Assess practical input sizes (will an $O(2^n)$ algorithm be feasible?)
- Discuss tradeoffs between different approaches

Let A and B be decision problems. We are given that A and B are problems in NP, and there exists a polynomial time reduction f from A to B. Assume that $P \neq NP$.

What a polynomial time reduction from A to B means is:

- If we could solve B efficiently (in polynomial time), then we could also solve A efficiently
- B is at least as hard as A

Given the information that:

1. A and B are both in NP
2. There exists a polynomial time reduction from A to B
3. $P \neq NP$ (as you've assumed)

We can conclude:

- If A is NP-complete, then B is also NP-complete
- If A is in P, B might be in P or it might be NP-complete
- B cannot be easier than A in terms of complexity

The existence of a polynomial time reduction from A to B only establishes a "lower bound" on B's difficulty - it must be at least as hard as A. It doesn't establish an "upper bound" that would place B in P. In fact, polynomial time reductions are the fundamental tool used to prove NP-completeness, by showing that a problem is at least as hard as known NP-complete problems.

DIVIDE AND CONQUER

Binary Search ($O(\log n)$) - dividing the search space in half until target element is found or not there

FFT: Polynomials evaluation in $O(n \log n)$ time by splitting even/odd terms ($T(m) = 2T(m/2) + cm$)

Merge Sort: divides arrays in half, sorts recursively, then merges in $O(n \log n)$ time

Integer Mult: Karatsuba's algo to multiple large integers faster than $O(n^2)$ traditional method, tc = for multiplying two n-digit number is $O(n^{\log_2 3})$.

Recurrence:

$$T(n) = aT\left(\frac{n}{b}\right) + f(n)$$

Master Theorem: Provides direct asymptotic solutions based on critical exponent $c^* = \log_b(a)$

1. Case 1: If $f(n)$ grows slower than n^{c^*} , then $T(n) = \Theta(n^{c^*})$
2. Case 2: If $f(n)$ grows at same rate as n^{c^*} , then $T(n) = \Theta(n^{c^*} \log n)$
3. Case 3: If $f(n)$ grows faster than n^{c^*} (with regularity condition), then $T(n) = \Theta(f(n))$

Example:

$$T(n) = 3T\left(\frac{n}{4}\right) + n, c^* = \log_4(3), \text{ and } f(n) = n, f(n) \text{ grows faster (case 3), so } T(n) = \Theta(n)$$

Recursion Tree Analysis: Visual method to understand how work is distributed across recursion levels, helps determine if most work happen at leaves, root or event across levels

Karatsuba trick

Split large no. into smaller parts and use math transformation. E.g. For multiplying 2323×2323 using Karatsuba: Split the number into two parts: $2323 = 23 \times 10^2 + 23 = 2300 + 23$ Since both numbers are the same, we calculate: $P1 = 23 \times 23 = 529$ (product of first parts) $P2 = 23 \times 23 = 529$ (product of second parts - the same as $P1$ in this case) $P3 = (23 + 23) \times (23 + 23) = 46 \times 46 = 2116$ (product of the sums) Calculate the middle term: $P3 - P1 - P2 = 2116 - 529 - 529 = 1058$

Convolutions:

Polynomials:

$O(n \log n)$, can be multiplied efficiently using FFT. Key steps: 1. Convert poly from coefficient to val representation by evaluating them at specific points (complex root of unity), 2. Multiply corresponding corresponding values point-wise (requiring $O(n)$ operations) 3. Convert back to coefficient from using the inverse FFT. uses DAC & properties of complex roots of unity to reduce computation comparing to $O(n^2)$ (when multiplying 2 polynomials)

Graph type that represents subproblem structure that represents DAC algorithm n-ary tree.

Greedy Algorithm

Key algorithms:

1. **Activity Selection:** Sort activities by finish time and select compatible activities greedily
 - Time Complexity: $O(n \log n)$ due to sorting
 - Application: Scheduling maximum number of non-overlapping activities
2. **Fractional Knapsack:** Sort items by value-to-weight ratio and take greedily
 - Time Complexity: $O(n \log n)$ due to sorting
 - Application: Maximizing value with weight constraint when items can be split
3. **Huffman Coding:** Build optimal prefix codes by repeatedly merging smallest-frequency nodes
 - Time Complexity: $O(n \log n)$ due to priority queue operations
 - Application: Data compression with variable-length codes
4. **Minimum Spanning Tree (Kruskal's):** Sort edges by weight and add if no cycle forms
 - Time Complexity: $O(E \log E)$ or $O(E \log V)$ due to sorting and union-find
 - Application: Finding minimum-cost tree connecting all vertices
5. **Minimum Spanning Tree (Prim's):** Grow tree by adding minimum-weight edge connected to current tree
 - Time Complexity: $O(E \log V)$ with binary heap, $O(E + V \log V)$ with Fibonacci heap
 - Application: Same as Kruskal's but different approach
6. **Dijkstra's Algorithm:** Greedily select vertex with minimum distance and update neighbors (suitable for weighted directed graph where all edge weights are non-negative)
 - Time Complexity: $O(E \log V)$ with binary heap
 - Application: Finding shortest paths from source to all vertices

Analysis Methods for Correctness

1. **Greedy Stays Ahead (Inductive):**
 - Show that at each step, the greedy solution is at least as good as any other solution
 - Prove that maintaining this "ahead" property throughout guarantees optimality
 - Example: In Activity Selection, prove selecting earliest-finishing activity maximizes remaining options
2. **Exchange Argument (Contradiction):**
 - Assume an optimal solution different from greedy solution
 - Show step-by-step exchanges that transform optimal to greedy without worsening solution
 - Example: In Fractional Knapsack, show any solution not in value/weight order can be improved

When proving greedy algorithm correctness, include:

1. **Clearly state the greedy choice:** Define exactly what "locally optimal" means in context
 - "We always select the activity with earliest finish time"
2. **Optimal substructure:** Show that once a greedy choice is made, remaining subproblem is similar
 - "After selecting activity i , we're left with same problem on compatible activities"
3. **Proof technique:** Use either "greedy stays ahead" or "exchange argument"

- "We'll prove by exchange argument that sorting by value/weight ratio is optimal"
- 4. **Base case and induction:** For inductive proofs, establish base case and show induction step
 - "Base: Selecting activity with earliest finish time is optimal for $n=1$ "
 - "Inductive step: If greedy is optimal for k activities, it's optimal for $k+1$ "
- 5. **Counterexample:** If greedy approach doesn't work generally, provide counterexample
 - "For non-fractional knapsack, greedy fails. Consider items $\{(10,5), (15,10), (7,5)\}$ "

Flow Networks

1. **Ford-Fulkerson Algorithm:** The Ford-Fulkerson algorithm finds the maximum flow in a flow network by repeatedly finding augmenting paths and increasing flow along these paths.
 - Initialize flow $f(u,v) = 0$ for all edges (u,v)
 - While there exists an augmenting path p from source s to sink t in the residual network:
 - Find the bottleneck capacity $cf(p) = \min\{cf(u,v) : (u,v) \text{ is in } p\}$
 - For each edge (u,v) in p :
 - $f(u,v) = f(u,v) + cf(p)$ (Increase flow)
 - $f(v,u) = f(v,u) - cf(p)$ (Decrease flow in reverse direction)
 - Return the final flow f

Correctness:

- Invariant: After each iteration, f is a valid flow (satisfies capacity constraints and flow conservation).
- Termination: If all capacities are integers, each augmentation increases flow by at least 1, so the algorithm terminates in at most $|f^*|$ iterations, where $|f^*|$ is the maximum flow value.
- Optimality: The algorithm terminates when no augmenting path exists, which occurs precisely when the max flow equals the min cut (by Max-Flow Min-Cut Theorem).

Time Complexity:

- $O(E \cdot |f^*|)$ where E is the number of edges and $|f^*|$ is the value of the maximum flow.
 - Note: This is pseudopolynomial since it depends on the flow value.
2. **Edmonds-Karp Algorithm:** A specific implementation of Ford-Fulkerson that uses BFS to find the shortest augmenting path (fewest number of edges) in each iteration. Same as Ford-Fulkerson, but use BFS to find augmenting paths.

Correctness:

- Same invariant and optimality as Ford-Fulkerson.
- Additionally, by always choosing the shortest path, we ensure the algorithm terminates in polynomial time.

Time Complexity:

- $O(V \cdot E^2)$ where V is the number of vertices and E is the number of edges.

- Proof sketch: Each edge can become critical (the bottleneck) at most V times, and finding an augmenting path with BFS takes $O(E)$ time.
3. **Capacity-Scaling Algorithm:** An improved version of Ford-Fulkerson that prioritizes augmenting paths with large capacities.
- Initialize flow $f(u,v) = 0$ for all edges (u,v)
 - Initialize Δ to the largest power of 2 not exceeding the maximum capacity
 - While $\Delta \geq 1$:
 - While there exists an augmenting path p with bottleneck capacity $\geq \Delta$:
 - Augment flow f along p
 - Set $\Delta = \Delta/2$
 - Return the final flow f

Correctness:

- Maintains the same invariants as Ford-Fulkerson.
- By scaling down Δ , we eventually consider all possible augmenting paths.

Time Complexity: $O(E^2 \log C)$ where C is the maximum capacity.

4. **Push-Relabel Algorithm:** Instead of finding augmenting paths, the Push-Relabel algorithm maintains a preflow (which may violate flow conservation) and gradually converts it to a valid maximum flow.
- Initialize height $h(s) = |V|$, $h(v) = 0$ for all other vertices
 - Initialize preflow $f(s,v) = c(s,v)$ for all edges from s , $f(u,v) = 0$ for all other edges
 - While there exists an overflowing vertex $v \neq s, t$:
 - If there is a residual edge (v,w) with $h(v) > h(w)$:
 - Push flow from v to w
 - Else:
 - Relabel v by setting $h(v) = 1 + \min \{h(w) : (v,w) \text{ is in the residual network}\}$
 - Return the final flow f

Correctness:

- Invariant: Heights always form a valid labeling.
- Termination: The number of relabel operations is bounded.
- Optimality: When the algorithm terminates, no vertex except possibly s has excess flow, ensuring we have a maximum flow.

Time Complexity:

- Generic implementation: $O(V^2E)$
- FIFO implementation: $O(V^3)$
- Highest-label implementation: $O(V^2\sqrt{E})$

5. **Dinic's Algorithm:** Uses level graphs to find multiple augmenting paths in one phase.

- Initialize flow $f(u,v) = 0$ for all edges (u,v)
- While BFS can create a level graph from s to t :
 - Use DFS to find blocking flows in the level graph
 - Augment the flow with these blocking flows

3. Return the final flow f

Correctness:

- Each phase increases the shortest path length from s to t .
- When no s - t path exists in the level graph, we've found the maximum flow.

Time Complexity:

- $O(V^2E)$ in general networks
- $O(E \cdot \min(V^{2/3}, E^{1/2}))$ in unit capacity networks
- $O(V \cdot E \cdot \log(V))$ in unit networks

6. **Bipartate Matching:** Using flow network by constructing a network where source connected to all vertices, (cap1), all vertices in first partition are connected to compatible vertices in the second partition (cap1), all vertices in second partition connected by to the sink (cap1)

1. Construct flow network with uni capacities
2. Run any maximum flow agl (Emonds-Karp, Ford)
3. Edges with flow = 1 between partitions indicate matches

Correctness:

- Integrality theorem guarantees integers flow
- Maximum flow corresponds exactly to a maximum matching
- Augmenting path in the flow corresponds to an augmenting path in the matching.

Time Complexity:

- Using a modified Ford-Fulkerson for bipartite graphs: $O(E\sqrt{V})$ where E is the number of edges and V is the number of vertices
- For specialized bipartite problems with structured constraints (like the intern allocation problem), a complexity of $O(n^2m^2)$ can be achieved by exploiting the problem structure

Analysis Methods for Flow Networks

1. **Residual Network Analysis:** When analyzing flow networks, the residual network is a crucial concept:
 - It represents the remaining capacity of the network
 - Enables identification of augmenting paths
 - Allows for flow cancellation and redirection
 - Provides a mechanism to prove optimality via cuts
2. **Cut Analysis:** A cut in a flow network is a partition of vertices into two sets S and T where $s \in S$ and $t \in T$:
 - The capacity of a cut is the sum of capacities of edges from S to T
 - The flow across a cut is the sum of flows on edges from S to T minus the sum of flows on edges from T to S
 - Min-cut is the cut with minimum capacity
3. **Max-Flow Min-Cut Theorem:** This fundamental theorem states that:
 - The maximum flow value equals the minimum cut capacity
 - Used to prove correctness of flow algorithms

- Provides a certificate of optimality

Writing for Correctness Proofs

1. Flow Conservation Invariant:
 - Show that at each step, the algorithm maintains a valid flow (except for push-relabel which uses preflows)
 - For each vertex $v \neq s, t$: $\text{inflow}(v) = \text{outflow}(v)$
2. Capacity Constraint Invariant:
 - Prove that flow never exceeds capacity: $0 \leq f(u,v) \leq c(u,v)$
3. Termination:
 - Prove the algorithm terminates in finite time
 - For Ford-Fulkerson, show that each augmentation increases the flow value
 - For other algorithms, bound the number of operations
4. Optimality via Max-Flow Min-Cut:
 - When no augmenting path exists, show that the set of vertices reachable from s in the residual network forms a min-cut
 - Prove the flow value equals this cut capacity, confirming optimality

Time Complexity Analysis

For Ford-Fulkerson:

- Each augmentation increases flow by at least 1 if capacities are integers
- Maximum number of iterations is $O(|f^*|)$, where $|f^*|$ is the maximum flow value
- Finding an augmenting path takes $O(E)$ time
- Total: $O(E \cdot |f^*|)$

For Edmonds-Karp:

- Key insight: The shortest path distance from s to v never decreases
- Each edge can become critical at most V times
- Total: $O(V \cdot E^2)$

For Push-Relabel:

- There are at most $2V^2$ relabel operations
- Number of saturating pushes is $O(V \cdot E)$
- Number of non-saturating pushes depends on implementation
- Various implementations have different complexities from $O(V^2E)$ to $O(V^2\sqrt{E})$

Practical Considerations

1. When to use each algorithm:
 - Ford-Fulkerson: Simple networks with small integer capacities
 - Edmonds-Karp: General networks with moderate size
 - Push-Relabel: Large dense networks
 - Dinic's: Large networks with unit capacities
2. Implementation techniques:
 - Use adjacency lists for sparse networks
 - Maintain current flows or just residual capacities (not both)

- Implement cancellation of flows implicitly through the residual network
- 3. Special cases:
 - For bipartite matching, flow algorithms achieve $O(E\sqrt{V})$ time complexity
 - For unit capacity networks, specialized algorithms are more efficient

Dynamic Programming:

Solving complex problems by breaking them into overlapping subproblems

Key Properties:

Optimal substructure: Optimal solution contains optimal solutions to subproblems

Overlapping subproblems: Same subproblems solved multiple times

Algorithm Design Process

- Define subproblems (carefully choose parameters)
- Develop recurrence relation (how smaller subproblems build larger ones)
- Identify base cases (trivial subproblems)
- Determine computation order (respecting dependencies)
- Construct final solution (from subproblem solutions)
- Analyse time complexity (usually #subproblems $\times$ time per subproblem)

Proving Correctness

- Justify that recurrence correctly relates subproblems
- Verify base cases are correct
- Confirm overall answer is correctly derived from subproblems
- Use induction to show optimal solutions to subproblems lead to optimal solution of original problem

Common DP Algorithms

One Parameter, Constant Recurrence:

Fibonacci/Hopscotch: $\text{opt}(i) = \text{opt}(i-1) + \text{opt}(i-2)$

Counts how many ways to reach a pos when you can take either 1 or 2 steps forwards, like counting different path options when hopping along a line. **TC** $O(n)$

One Parameter, Linear Recurrence

Longest Increasing Subsequence: $\text{opt}(i) = 1 + \max\{\text{opt}(j)\}$ for $j < i$ where $A[j] < A[i]$.

Finds the longest chain of numbers that keep getting bigger (not necessarily next to each other) by checking each pos and extending previous chain. **TC** $O(n^2)$

Making Change: $\text{opt}(i) = 1 + \min\{\text{opt}(i-v)\}$ for all coins $v \leq i$

Figures out the fewest coins needed to make certain amount by trying each coin type and building on previously calculated smaller amounts. **TC** $O(nc)$ - **pseudopolynomial**

Two Parameters, Constant Recurrence

0-1 Knapsack: $\text{opt}(i,k) = \max[v_i + \text{opt}(i-w_i, k-1), \text{opt}(i, k-1)]$

Helps you pack a bag optimally when you can either take or leave each item, by checking if adding the current item is better than leaving it out. **TC $O(nC)$**

Longest Common Subsequence:

$\text{opt}(i,j) = \{$
 $1 + \text{opt}(i-1, j-1)$ if $A[i]=B[j]$
 $\max[\text{opt}(i-1, j), \text{opt}(i, j-1)]$ otherwise
 $\}$

Finds the longest string of characters that appear in the same order in two strings, like finding what parts of two DNA sequences match up. **TC $O(nm)$**

Two Parameters, Linear Recurrence

Matrix Chain Multiplication: $\text{opt}(i,j) = \min\{\text{opt}(i,k) + \text{opt}(k+1,j) + \text{cost}\}$ for $i < k < j$

Figures cheapest way to multiply a bunch of matrices by trying different ways to put parentheses around them. **TC $O(n^3)$**

Graph Algorithms

DAG Shortest Path: $\text{opt}(t) = \min\{\text{opt}(v) + w(v,t)\}$ for all edges (v,t)

Finds the quickest route that one-way maze with no loops by working through locations in a special order. **TC $O(V+E)$**

Bellman-Ford: $\text{opt}(i,t) = \min\{\text{opt}(i-1,v) + w(v,t)\}$ for all edges (v,t)

Calculates shortest paths even when some routes might have negative distances, checking paths with increasing no. of steps **TC $O(VE)$**

Floyd-Warshall: $\text{opt}(i,j,k) = \min[\text{opt}(i,j,k-1), \text{opt}(i,k,k-1) + \text{opt}(k,j,k-1)]$

Finds all shortest paths between every pair of points by gradually allowing more stopover locations. **TC: $O(V^3)$**

String Matching

Knuth-Morris-Pratt: Uses failure function to skip redundant comparisons

Searches for a pattern in text without backtracking, by remembering partial matches so you don't have to start over when there's a mismatch. **TC $O(n+m)$**

Recovery of Solutions

Store predecessor information along with optimal values. Backtrack through table to reconstruct solution

Implementation Approaches

Bottom-up (Tabulation): Iteratively build solutions from smallest to largest

Top-down (Memorisation): Recursive with caching of results

Memory Optimisation

Often only need most recent computed values, not entire table

Can reduce space complexity (e.g., using only two rows instead of complete table)

Knuth-Morris-Pratt Algorithm

1. Preprocessing: Build a partial match table (called 'failure function')
2. Search: Use this table to skip charas efficiently when a mismatch occurs

To prove that $\text{\texttt{Dandelion}}$ is a NP-complete, we need to show 2 things:

- \begin{enumerate}
- \item $\text{\texttt{Dandelion}}$ is in NP
- \item $\text{\texttt{Dandelion}}$ is NP-hard by reducing a known NP-complete problem to it
- \end{enumerate}

To show that $\text{\texttt{Dandelion}}$ is in NP. A problem is in NP if we can verify a solution in polynomial time. For $\text{\texttt{Dandelion}}$ problem, given a certificate consisting of the vertices that form the $\text{\texttt{Dandelion}}$ sub graph of size at least k , we can verify in polynomial time that these vertices indeed form a path of length $k-1$ followed by k additional vertices connected to one endpoint of the path. Specifically, we check that the first k vertices form a path and that the leftover k vertices are all connected to the last vertex of the path. this verification can be done in $O(K^2)$ time, which is polynomial.

To prove that $\text{\texttt{Dandelion}}$ is a NP-hard, we will reduce the Hamiltonian Path problem to it. The Hamiltonian Path problem asks whether there exists a path in a graph that visits each vertex exactly once. Given an instance of the Hamiltonian Path problem with graph $G = (V, E)$ and n vertices, we construct an instance of the $\text{\texttt{Dandelion}}$ problem as follows:

- \begin{itemize}
- \item We create a new graph G' by adding n vertices $v_1, v_2, \dots, v_n$ to G and connecting each of these new vertices to a single vertex $u \in V$
- \item We then set $k = n$ for our $\text{\texttt{Dandelion}}$ instance. We claim that G has a Hamiltonian path ending at vertex u if and only if G' has a $\text{\texttt{Dandelion}}$ sub graph of size n .
- \end{itemize}

For the forward direction, if G has a Hamiltonian path $SP = (u_1, u_2, \dots, u_n)$ ending at vertex u , then in G' , we can form a $\text{\texttt{Dandelion}}$ of size n by using this path and connecting the n new vertices $v_1, v_2, \dots, v_n$ as petals to u , the endpoint of the path. This will give us $\text{\texttt{Dandelion}}$ of size n in G'

On the other hand, suppose G' has a $\text{\texttt{Dandelion}}$ of size n . This $\text{\texttt{Dandelion}}$ consists of a path of length $n-1$ followed by n additional vertices connected to one endpoint of the path. Since the n new vertices are only connected to vertex u and not to each other, they cannot form part of the path. Therefore, they must serve as the n petals of the $\text{\texttt{Dandelion}}$, and u must be the endpoint of the path to which they connect. The remaining

path of length $n-1$ must consist entirely of vertices from G , and together with u , they form a path visiting all n vertices in G , which means it is a Hamiltonian path in G ending at u .

The reduction can be performed in polynomial time, as creating G' involves adding n new vertices and at most n^2 new edges, which can be done in $O(n^2)$ time.

Therefore, since $\text{problem}\{\text{Dandelion}\}$ is in NP and is NP-hard, we can conclude that $\text{problem}\{\text{Dandelion}\}$ is NP-complete.

Finding a **Hamiltonian path** is more complex than just using standard DFS. Let me explain the key differences:

A standard DFS algorithm alone isn't sufficient to find a Hamiltonian path because:

1. A Hamiltonian path must visit every vertex exactly once
2. Standard DFS may revisit vertices and doesn't track if all vertices have been visited

To solve the Hamiltonian path problem, you need:

1. A backtracking algorithm (which can be implemented using DFS as the search strategy)
2. Additional constraints to ensure:
 - * Each vertex is visited exactly once
 - * The path includes all vertices

The correct approach is typically:

- * Use DFS with backtracking
- * Maintain a visited array to avoid revisiting vertices
- * Track the path length to ensure all vertices are included
- * When a dead end is reached, backtrack and try different paths

The Hamiltonian path problem is NP-complete, meaning there's no known polynomial-time algorithm.

The backtracking solution has exponential time complexity $O(n!)$, making it impractical for large graphs.

For small graphs, this backtracking approach works, but for larger instances, heuristic algorithms or approximation methods are typically used instead.

Guidelines for Calculating Flow Network Time Complexity

When analyzing time complexity for flow network algorithms, follow these steps:

1. Identify the network parameters:
 - * V = number of vertices
 - * E = number of edges
 - * $|f|$ = maximum flow value
2. Know your algorithm bounds:
 - * Ford-Fulkerson: $O(E \cdot |f|)$ - depends on max flow value
 - * Edmonds-Karp: $O(V \cdot E^2)$ general bound, but can be tightened to $O(E \cdot |f|)$ when analyzing specific problems
3. Consider problem-specific constraints:
 - * Is there a bound on the maximum flow? (Like r in your problem)
 - * Are there specific network structures that limit paths?
 - * Are capacities integers or unit capacities?
4. Express in terms of the required variables:

- * Translate V and E into your problem variables (r , c , n , etc.)
 - * Apply any known bounds on maximum flow
5. Simplify the expression:
- * Remove lower-order terms
 - * Combine like terms to get the final bound

Divide and Conquer The subproblem structure is best represented by an n -ary tree

Greedy Algorithms The subproblem structure is best represented by a linear chain or path

Flow networks The subproblem structure is best represented by a layered graph or residual network

Dynamic Programming The subproblem structure is best represented by a directed acyclic graph (DAG):

NP-Complete Problems The subproblem structure is best represented by a state space graph or search tree:

Problem 7 (15 marks) (continued)

Once again, they both have correct algorithms. Amy's algorithm is:

Perform a single linear scan of the first row, to find the smallest number and its index i . Then, perform a linear scan of the second row, but starting from $i + 1$. We continue in this way for each row, starting the scan for each row one index after the position of the smallest element in the previous row.

And Bruno's:

First, perform a linear scan to find the smallest element (and its index i) in row $\lfloor \frac{n}{2} \rfloor$. Then, use the same method recursively to find the smallest element for each row in the subgrid from $(1, 1)$ to $(\lfloor \frac{n}{2} \rfloor - 1, i - 1)$ (i.e. the subgrid of all elements above and to the left of the smallest element in row $\lfloor \frac{n}{2} \rfloor$), and recursively find the smallest element for each row in the subgrid from $(\lfloor \frac{n}{2} \rfloor + 1, i + 1)$ to (n, m) (i.e. the subgrid of all elements below and to the right).

Decide if Amy's algorithm is **asymptotically** better than, worse than, or the same as Bruno's in the *worst case*. Justify your answer, including an analysis of the time complexity of each algorithm.

Hint. The Master Theorem is only applicable for recurrences of a single variable. You may wish to consider the tree structure of the recursion instead.

Solution.

- (a) Amy's algorithm runs in $O(n^2)$, since in the worst case, she has to check each pair in constant time, before declaring no. Thus, Amy's algorithm runs in $\Theta(n^2)$ in the worst case.

Bruno's algorithm satisfies the recurrence $2T(n/2) + O(n^2)$, since there are at most $\lceil n^2/4 \rceil$ comparisons at the combine step, and there are two subproblems half the size of the original problem. We fall into Case 3 of the Master Theorem as the critical exponent is 1.

Choose $1/2 \leq k < 1$. Then, note that $kf(n) \geq n^2/2 = 2(n/2)^2 = af(n/2)$, so the regularity condition is satisfied. Thus, Bruno's algorithm runs in $\Theta(n^2)$ in the worst case.

Hence, their algorithms are asymptotically the same.

- (b) In the worst case, Amy searches through $(m - n)n + n(n - 1)/2$ many elements (all the smallest elements are backloaded as far as possible). Clearly, her algorithm runs in $\Theta(mn)$ in the worst case, since $n < m$.

Bruno's algorithm has a recursion tree of height $\log n$, and at each level, we search the

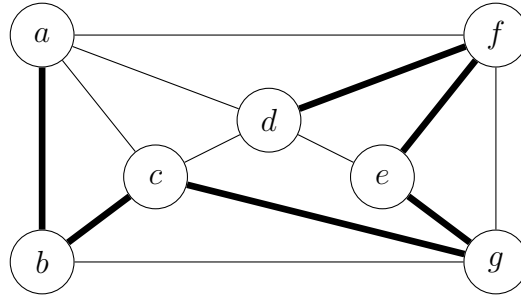
entire middle row in the worst case, in $\Theta(m)$ time.

Thus, we have that Bruno's time complexity is $O(m \log n)$, which we observe is a factor of $n/\log n$ faster than Amy's algorithm.

Problem 8 (15 marks)

In a graph $G(V, E)$, a *simple path* is a sequence of vertices which does not include any vertex twice, and each pair of adjacent vertices in the sequence is joined by an edge in E . The length of a path is given by the number of edges it uses.

A *Hamiltonian path* is a simple path that visits every vertex of the graph.



In this example, the sequence $[a, b, c, g, e, f, d]$ is a Hamiltonian path.

We define the HAMPATH problem on directed graphs as follows:

HAMPATH(G): Given an unweighted directed graph $G = (V, E)$, does G contain a Hamiltonian path?

Correspondingly, we also define the longest path problem (LPP) as follows:

LPP(G, k): Given an unweighted directed graph $G = (V, E)$, does G contain a simple path of length at least k ?

- (a) **[4 marks]** You are given an instance of the LPP problem. You are also given a vertex sequence $P = [v_1, v_2, \dots, v_t]$. Outline a polynomial time algorithm that determines if P is a simple path with length at least k in G .

Briefly justify that your algorithm is correct, and runs in polynomial time.

- (b) **[6 marks]** Describe a polynomial time reduction from HAMPATH to LPP.

- (c) **[3 marks]** Hence, explain why LPP is NP-Complete.

Hint. You may assume that HAMPATH is NP-Complete.

- (d) **[2 marks]** Suppose LPP is computationally *easy* (ie. able to be solved in polynomial time) on a particular set S of directed graphs. Explain why HAMPATH is also easy on all graphs in S .

If you did not complete part (a) and/or (b), you may assume that both of these parts have a solution in order to answer parts (c) and (d).

Solution.

- (a) (Sketch.) Check that for every $1 \leq i \leq t - 1$, that $(v_i, v_{i+1}) \in E$. Then, verify that all

the v_i are distinct (by maintaining a set, or otherwise, and verifying membership before adding to the set). Then, if $|P| \geq k$, this is a simple path with length at least k in G . Otherwise this is not a simple path with length at least k in G .

This runs in polynomial time as verifying that each exists is in the worst case quadratic in $|E|$, verifying that all the v_i are distinct runs is also in the worst case quadratic in $|V|$. Checking $|P|$ is also worst case linear in $|V|$.

- (b) G has a Hamiltonian path iff G has a simple path of length at least $|V| - 1$. Since we preserve the graph, this clearly runs in polynomial time.

($\implies$) Suppose G has a Hamiltonian path P . Since P visits every vertex exactly once, this path has length $|V| - 1$, thus G also has a simple path of length at least $|V| - 1$.

($\impliedby$) Suppose G has a path of length at least $|V| - 1$. Since the path is simple, it necessarily visits every vertex, and thus this is also a Hamiltonian path in G .

- (c) From part (a), we have that LPP is in class NP. From part (b), since HAMPATH is in NP-C, that LPP is in NP-H. Therefore, we have that LPP is in NP-C.
- (d) The reduction in (b) is easy, so for all $G \in S$, we can reduce the instance of HAMPATH to the corresponding instance of LPP before solving, which is known to be easy with input G . Thus, this runs HAMPATH is easy for all $G \in S$.

Part III: Long Response Questions

Problem 9 (20 marks)

A startup company is planning on producing gadgets. There is a one-time cost of D dollars to design the gadget.

There are n customers who each want to buy at most one gadget. The i th customer will pay at most $C[i]$ dollars. Once the company chooses a price P , they can sell at that price to each customer i for whom $C[i] \geq P$.

The company makes a profit if the total amount they make from selling the gadgets is more than the design cost. Design an $O(n \log n)$ algorithm that finds the **fewest gadgets** the company needs to sell in order to make a profit, or determines that it is impossible to make a profit.

Example. If we have $n = 7$ interested customers, with $C = [6, 13, 12, 6, 14, 3, 15]$ and $D = 30$.

If we price the gadget at 11 dollars, we could sell to customers 2, 3, and 5. The total profit will be:

- $3 \times 11 = 33$ dollars from selling 3 gadgets,
- -30 dollars to design the gadget,
- $= 3$ dollars profit.

There is no way to make a profit by selling less than 3 gadgets, so the answer is 3.

Input and Output. The input for the algorithm is an integer n , an array C containing n positive integers, and an integer D . The array C is in no particular order.

If it is possible to make a profit, then the output is a single integer, which is the fewest gadgets the company can sell to make a profit. Otherwise, your algorithm must report 'not possible'.

Solution. First, sort C in descending order. If we are going to sell to i customers, we should sell to the i who are willing to pay the most, and charge them as much as possible - so we should charge $C[i]$, and we will make $i \times C[i] - D$ dollars. Evaluate $i \times C[i] - D$ for each i , and the minimum number of gadgets that need to be sold is the first i for which this is positive. If it is negative or zero for all i , it is impossible to make a profit.

Suppose there is a solution which sells to k people and makes a profit, where the greedy algorithm sells to more than k people. Then, we can assume the solution sells to the people with the k highest prices - otherwise, we can exchange the people in the solution for the k highest people and this produces at least that amount of profit. However, our greedy algorithm would have considered this group of the i highest people, and found that it would not make a profit. Therefore, the greedy algorithm sells the fewest number of gadgets possible.

Sorting takes $O(n \log n)$, then we evaluate the profit in constant time for each i , so the algorithm is $O(n \log n)$.

Solution.

(Sketch.) Let $\text{HASWALK}(i, v)$ denote whether a walk of weight i from anywhere to v exists.

By definition, $\text{HASWALK}(i, v) = \text{FALSE}$ for all $i < 0, v \in V$, and $\text{HASWALK}(0, v) = \text{TRUE}$ for all $v \in V$.

Do not write your answers on this booklet.

15

Then, our recurrence is given by $\text{HASWALK}(i, v) = \bigvee_{u: (u,v) \in E} \text{HASWALK}(i - w(u, v), u)$.

Our order of computation is in increasing order of i and any order of v , to fulfil all the subproblem dependencies.

Our time complexity is $O(km)$, since we evaluate over each edge once at every walk weight from 0 to k inclusive, when filling out our table of results of size $k \times n$.

Our final answer is $\bigvee_{v \in V} \text{HASWALK}(k, v)$.

Our base cases are necessarily correct, since the only walks of weight 0 are a singular vertex itself (since all edge weights are positive), and no walks of negative weight exist for the same reason.

Then, notice that every walk of weight i to a vertex v must be constructed from a walk of length $i - w(u, v)$ from a vertex adjacent to v (ie. appending the edge to a walk with the remaining weight) so the existence of a walk depends only on the existence of at least one corresponding 'subwalk' from an adjacent vertex. No other walks can be extended that contribute to the current subproblem, so our subproblems must respect the recurrence.

Do not write your answers on this booklet.

14

Problem 10 (20 marks)

You have a connected, weighted graph $G = (V, E)$ with n vertices and m edges. For any two vertices u and v , the weight of the edge from u to v (if it exists) is given by $w(u, v)$. The weights are all positive integers. You are also given a non-negative integer k .

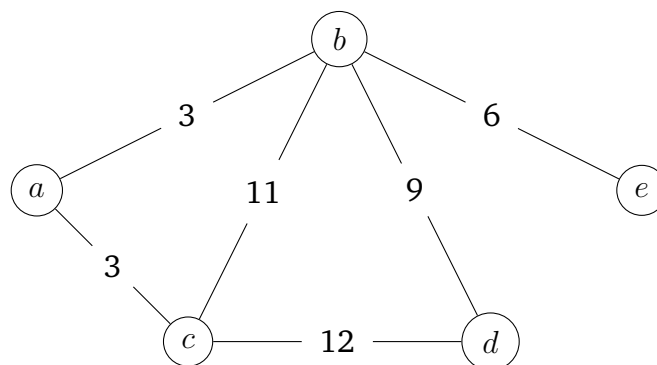
A *walk* is a sequence of vertices such that each vertex in the sequence has an edge to the next one. In other words, it is a path through the graph that may include the same vertex multiple times.

Design an $O(km)$ algorithm that determines whether or not there is a walk in the graph with a weight of exactly k . The walk can start at any vertex and end at any vertex (including the vertex where it started).

You must justify the correctness of your algorithm, and you must justify that it has the required time complexity.

Example. Suppose G is the graph shown below.

- If $k = 26$, then the answer is yes. One example of a walk with a weight of 26 is $[d, b, c, a, b]$.
- If $k = 10$, the answer is no. There is no walk in the graph with a weight of 10.



Input and Output. The input to the algorithm is a connected, weighted, undirected graph G with n vertices and m edges, and a positive integer k . The edge weights in G are all positive integers. You may assume G is given as either an adjacency matrix or adjacency list.

The output is ‘yes’ if G contains a walk with a total weight of exactly k , and ‘no’ otherwise. Your algorithm does not need to find a specific walk; it only needs to determine whether or not one exists.

Solution.

(Sketch.) Let $\text{HASWALK}(i, v)$ denote whether a walk of weight i from anywhere to v exists.

By definition, $\text{HASWALK}(i, v) = \text{FALSE}$ for all $i < 0, v \in V$, and $\text{HASWALK}(0, v) = \text{TRUE}$ for all $v \in V$.

Then, our recurrence is given by $\text{HASWALK}(i, v) = \bigvee_{u: (u, v) \in E} \text{HASWALK}(i - w(u, v), u)$.

Our order of computation is in increasing order of i and any order of v , to fulfil all the subproblem dependencies.

Our time complexity is $O(km)$, since we evaluate over each edge once at every walk weight from 0 to k inclusive, when filling out our table of results of size $k \times n$.

Our final answer is $\bigvee_{v \in V} \text{HASWALK}(k, v)$.

Our base cases are necessarily correct, since the only walks of weight 0 are a singular vertex itself (since all edge weights are positive), and no walks of negative weight exist for the same reason.

Then, notice that every walk of weight i to a vertex v must be constructed from a walk of length $i - w(u, v)$ from a vertex adjacent to v (ie. appending the edge to a walk with the remaining weight) so the existence of a walk depends only on the existence of at least one corresponding ‘subwalk’ from an adjacent vertex. No other walks can be extended that contribute to the current subproblem, so our subproblems must respect the recurrence.